

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	7
1.2 Описание выходных данных.....	7
2 МЕТОД РЕШЕНИЯ.....	9
3 ОПИСАНИЕ АЛГОРИТМОВ.....	12
3.1 Алгоритм конструктора класса Sensor.....	12
3.2 Алгоритм метода setValues класса Sensor.....	12
3.3 Алгоритм метода print класса Sensor.....	13
3.4 Алгоритм метода setPrivateValue класса Sensor.....	13
3.5 Алгоритм конструктора класса SmartSensor.....	14
3.6 Алгоритм метода setValues класса SmartSensor.....	14
3.7 Алгоритм метода print класса SmartSensor.....	14
3.8 Алгоритм функции main.....	15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	17
5 КОД ПРОГРАММЫ.....	21
5.1 Файл main.cpp.....	21
5.2 Файл Sensor.cpp.....	22
5.3 Файл Sensor.h.....	22
5.4 Файл SmartSensor.cpp.....	23
5.5 Файл SmartSensor.h.....	23
6 ТЕСТИРОВАНИЕ.....	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26

1 ПОСТАНОВКА ЗАДАЧИ

В системе мониторинга используется **базовый датчик**, который хранит внутренний калибровочный параметр и измеряемое значение. На основе базового датчика разрабатывается **умный датчик**, который наследует свойства базового датчика, но имеет собственные поля и методы с теми же именами.

Требуется разработать систему, которая

- демонстрирует создание объекта производного класса;
- демонстрирует сохранение доступа к открытым элементам базового класса;
- демонстрирует сокрытие одноимённых элементов базового класса;
- демонстрирует однозначное обращение к элементам базового и производного классов.

Базовый класс должен содержать:

- в закрытом доступе:
 - одно свойство целого типа;
 - метод с одним параметром, который устанавливает значение закрытого свойства равным утроенному значению параметра.
- в открытом доступе:
 - одно свойство целого типа;
 - параметризованный конструктор с двумя параметрами(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
 - метод с двумя параметрами для изменения значений свойств(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
 - метод вывода значений свойств в формате "Sensor: <закрытое

свойство> <открытое свойство>"

Производный класс должен наследоваться от базового класса таким образом, чтобы к открытым элементам исходного класса сохранить открытый доступ и у него:

- в закрытом доступе:
 - о одно свойство целого типа с тем же именем, что и закрытое свойство базового класса.
- в открытом доступе:
 - о одно свойство целого типа с тем же именем, что и открытое свойство базового класса;
 - о параметризованный конструктор (1 параметр - установка закрытого свойства, 2 параметр - установка открытого свойства);
 - о метод изменения значений свойств с двумя параметрами;
 - о метод вывода значений свойств в формате: "SmartSensor: <закрытое свойство производного класса> <открытое свойство производного класса>"

Алгоритм работы программы

1. Объявить две целочисленные переменные.
2. Ввести значения двух переменных.
3. Создать объект производного класса.
4. Вывести значения свойств базовой части объекта.
5. Вывести значения свойств производной части объекта.
6. Ввести одно целое число.
7. Присвоить это значение открытому свойству производного класса.
8. Присвоить это значение открытому свойству базового класса.
9. Вывести значения свойств производного объекта.
10. Вывести значения свойств базового объекта.

11. Ввести два целых числа a и b .
12. Если первое число больше нуля:
 - 12.1. вызвать метод установки свойств производного объекта с параметрами $a+1$ и $b+1$;
 - 12.2. вызвать метод установки свойств базового объекта с параметрами $a-1$ и $b-1$;
 - 12.3. вывести значения производного объекта;
 - 12.4. вывести значения базового объекта.
13. Иначе:
 - 13.1. вызвать метод установки свойств базового объекта с параметрами $a+1$ и $b+1$;
 - 13.2. вызвать метод установки свойств производного объекта с параметрами $a-1$ и $b-1$;
 - 13.3. вывести значения базового объекта;
 - 13.4. вывести значения производного объекта.
14. Завершить работу программы.

1.1 Описание входных данных

Пример входных данных:

```
3 4
5
2 3
```

1.2 Описание выходных данных

Пример выходных данных:

```
Sensor: 9 4
SmartSensor: 3 4
SmartSensor: 3 5
```

Sensor: 9 5
SmartSensor: 3 4
Sensor: 3 2

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект smartSensor класса SmartSensor предназначен для хранения и обработки данных умного датчика с возможностью сокрытия одноимённых полей базового класса;
- функция main для организации работы программы: ввода данных, создания объекта, вызова методов изменения свойств и вывода значений;
- оператор :: - однозначное обращение к элементам базового класса при их сокрытии одноимёнными элементами производного класса.

Класс Sensor:

- свойства/поля:
 - поле хранение закрытого значения датчика:
 - наименование — privateValue;
 - тип — цел;
 - модификатор доступа — private;
 - поле хранение открытого значения датчика:
 - наименование — publicValue;
 - тип — цел;
 - модификатор доступа — public;
- функционал:
 - метод Sensor — конструктор для инициализации закрытого и открытого свойств;
 - метод setValues — изменение обоих свойств датчика;
 - метод print — вывод значений свойств в формате "Sensor: <закрытое> <открытое>";
 - метод setPrivateValue — установка закрытого свойства равным

утроенному значению параметра.

Класс SmartSensor:

- свойства/поля:
 - поле сокрытие одноимённого закрытого поля базового класса:
 - наименование — privateValue;
 - тип — цел;
 - модификатор доступа — private;
 - поле сокрытие одноимённого открытого поля базового класса:
 - наименование — publicValue;
 - тип — цел;
 - модификатор доступа — public;
- функционал:
 - метод SmartSensor — конструктор для инициализации собственных свойств производного класса;
 - метод setValues — изменение собственных свойств производного класса;
 - метод print — вывод значений свойств в формате "SmartSensor: <закрытое> <открытое>".

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Sensor			хранение калибровочного параметра (закрытого, хранится в утроенном виде) и открытого измеряемого значения)	
		SmartSensor	public		2
2	SmartSensor			демонстрация сокрытия одноимённых полей базового класса и независимого управления собственными параметрами	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса *Sensor*

Функционал: конструктор для инициализации закрытого и открытого свойств.

Параметры: цел *param1* - значение для закрытого свойства, цел *param2* - значение для открытого свойства.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса *Sensor*

№	Предикат	Действия	№ перехода
1		вызвать собств. метод <i>setValues</i> с парам. <i>param1</i> и <i>param2</i>	Ø

3.2 Алгоритм метода *setValues* класса *Sensor*

Функционал: изменение обоих свойств датчика.

Параметры: цел *param1* - значение для закрытого свойства, цел *param2* - значение для открытого свойства.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода *setValues* класса *Sensor*

№	Предикат	Действия	№ перехода
1		вызвать <i>setPrivateValue</i> с параметром <i>param1</i>	2

№	Предикат	Действия	№ перехода
2		присвоить publicValue значение param2	Ø

3.3 Алгоритм метода print класса Sensor

Функционал: вывод значений свойств в формате "Sensor: <закрытое> <открытое>".

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода print класса Sensor

№	Предикат	Действия	№ перехода
1		вывести на консоль "Sensor: ", privateValue, " ", publicValue, std::endl	Ø

3.4 Алгоритм метода setPrivateValue класса Sensor

Функционал: установка закрытого свойства равным утроенному значению параметра.

Параметры: цел value - новое значение закрытого свойство, которое будет утроено.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода setPrivateValue класса Sensor

№	Предикат	Действия	№ перехода
1		присвоить privateValue результат операции value * 3	Ø

3.5 Алгоритм конструктора класса SmartSensor

Функционал: конструктор для инициализации собственных свойств производного класса.

Параметры: цел param1 - значение для закрытого свойства, цел param2 - значение для открытого свойства.

Алгоритм конструктора представлен в таблице 6.

Таблица 6 – Алгоритм конструктора класса SmartSensor

№	Предикат	Действия	№ перехода
1		вызвать конструктор Sensor с параметрами param1, param2	2
2		вызвать собствен. метод setValues с параметрами param1, param2	Ø

3.6 Алгоритм метода setValues класса SmartSensor

Функционал: изменение собственных свойств производного класса.

Параметры: цел param1 - значение для закрытого свойства, цел param2 - значение для открытого свойства.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода setValues класса SmartSensor

№	Предикат	Действия	№ перехода
1		присвоить privateValue значение param1	2
2		присвоить publicValue значение param2	Ø

3.7 Алгоритм метода print класса SmartSensor

Функционал: вывод значений свойств в формате "SmartSensor: <закрытое> <открытое>".

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *print* класса *SmartSensor*

№	Предикат	Действия	№ перехода
1		вывести на консоль "SmartSensor: ", privateValue, " ", publicValue, std::endl	Ø

3.8 Алгоритм функции *main*

Функционал: организация работы программы: ввод данных, создание объекта, вызов методов изменения свойств и вывод значений.

Параметры: нет.

Возвращаемое значение: цел число - код возврата программы.

Алгоритм функции представлен в таблице 9.

Таблица 9 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявл. цел. var1, var2	2
2		ввод значений var1, var2	3
3		создание объекта smartSensor класса SmartSensor с парам. var1 и var2	4
4		вызов метода smartSensor::Sensor.print	5
5		вызов метода smartSensor.print	6
6		объявл. цел. number	7
7		ввод значения number	8
8		присвоить smartSensor.publicValue значение number	9
9		присвоить smartSensor::Sensor.publicValue	10

№	Предикат	Действия	№ перехода
		значение number	
10		вызов метода smartSensor.print	11
11		вызов метода smartSensor::Sensor.print	12
12		объявл. цел. a, b	13
13		ввод значений a, b	14
14	a > 0	вызов метода smartSensor.setValues с парам. a + 1, b + 1	15
		вызов метода smartSensor::Sensor.setValues с парам. a + 1, b + 1	18
15		вызов метода smartSensor::Sensor.setValues с парам. a - 1, b - 1	16
16		вызов метода smartSensor.print	17
17		вызов метода smartSensor::Sensor.print	21
18		вызов метода smartSensor.setValues с парам. a - 1, b - 1	19
19		вызов метода smartSensor::Sensor.print	20
20		вызов метода smartSensor.print	21
21		возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-4.

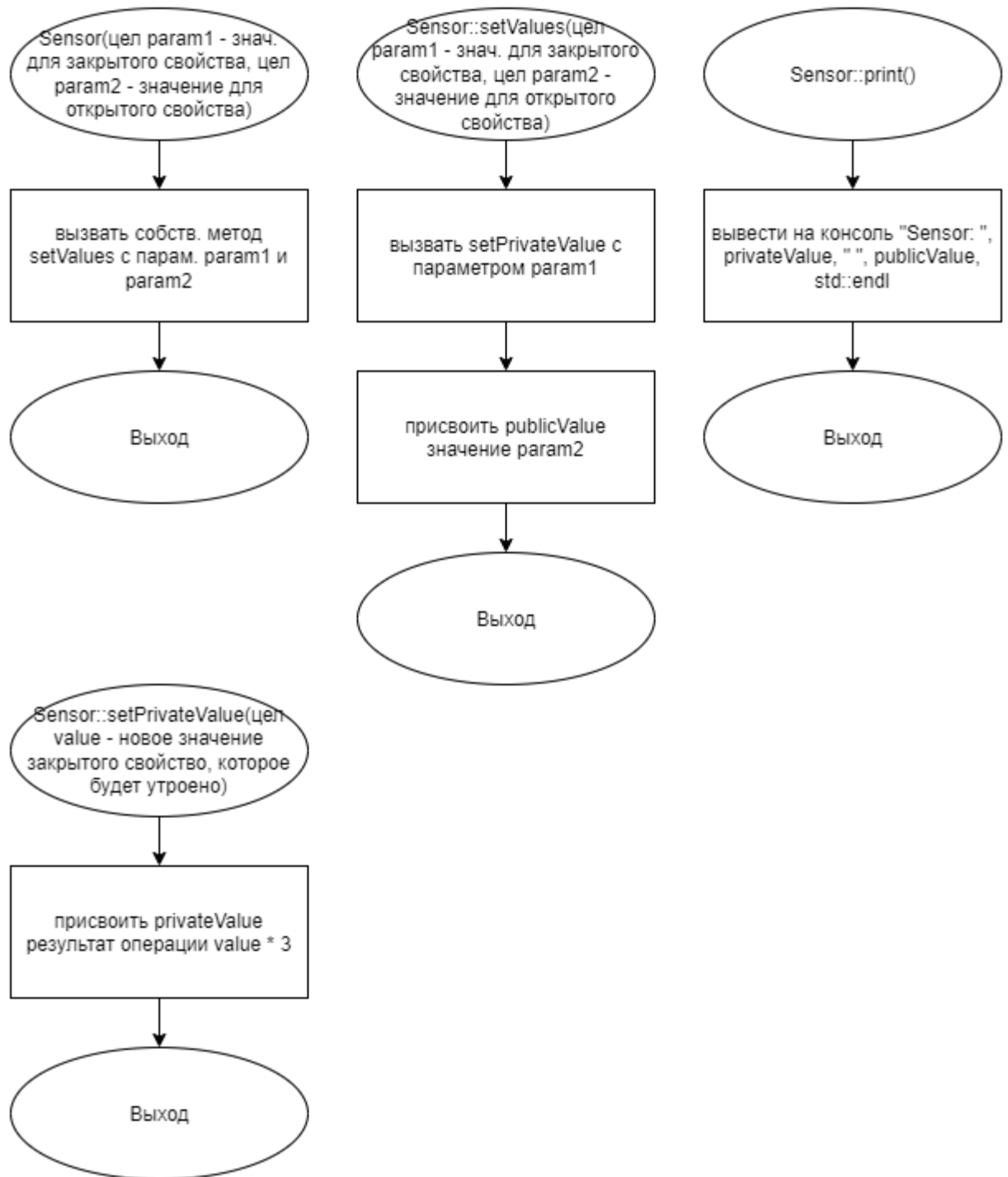


Рисунок 1 – Блок-схема алгоритма

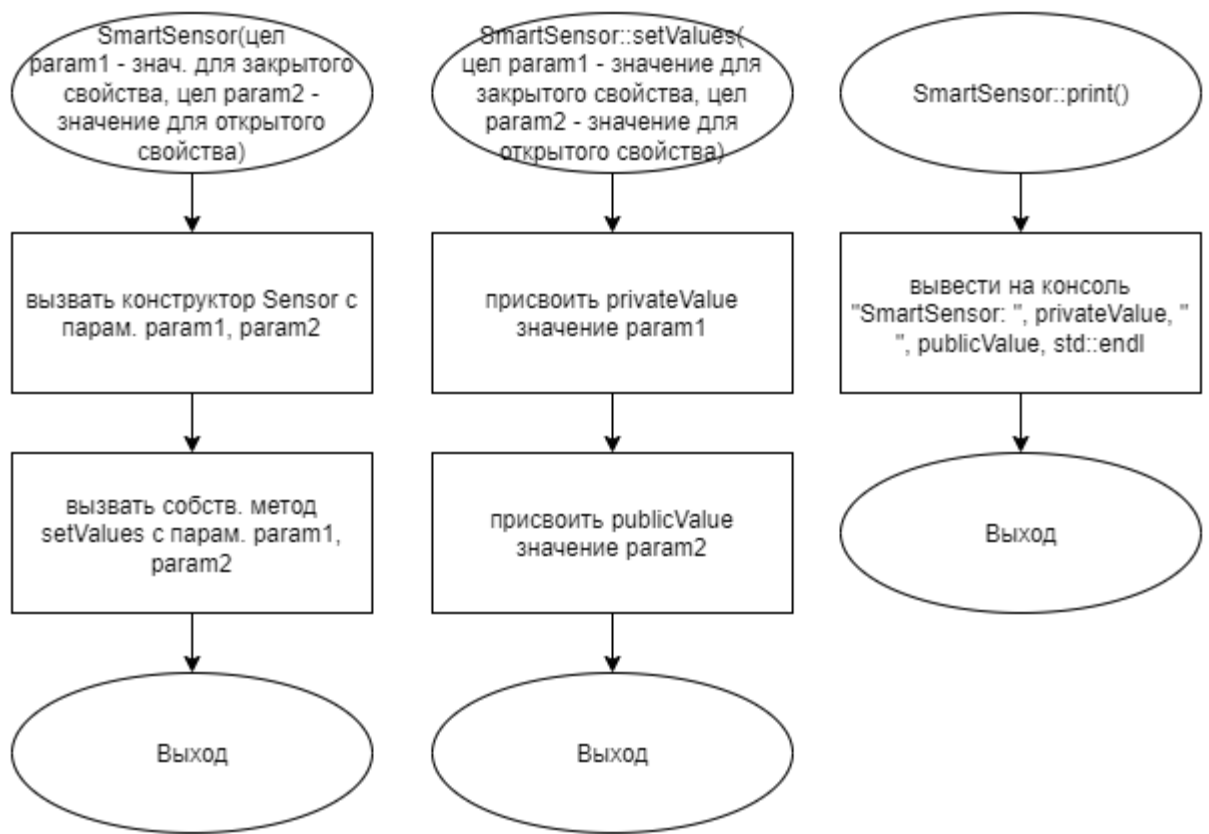


Рисунок 2 – Блок-схема алгоритма

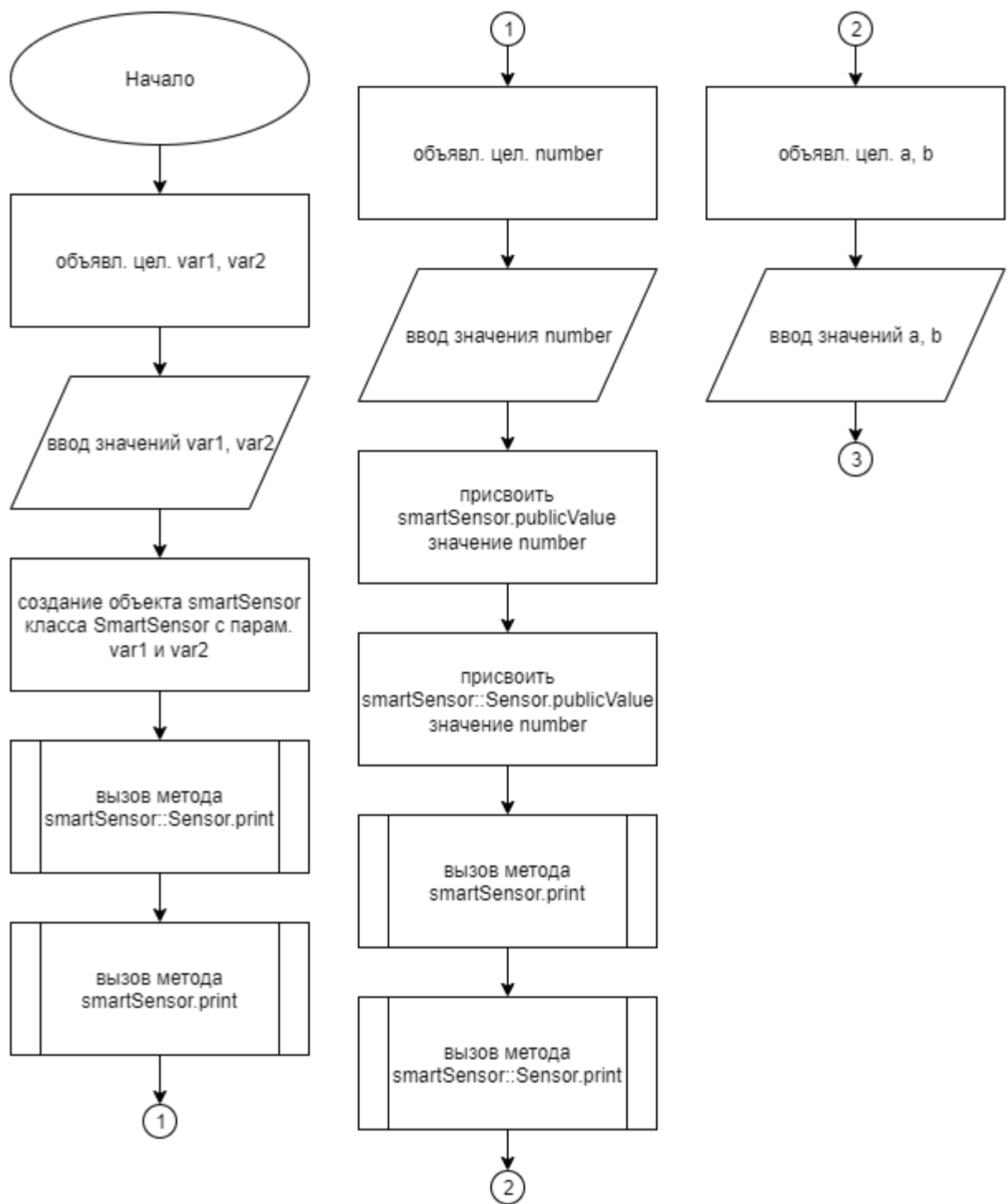


Рисунок 3 – Блок-схема алгоритма

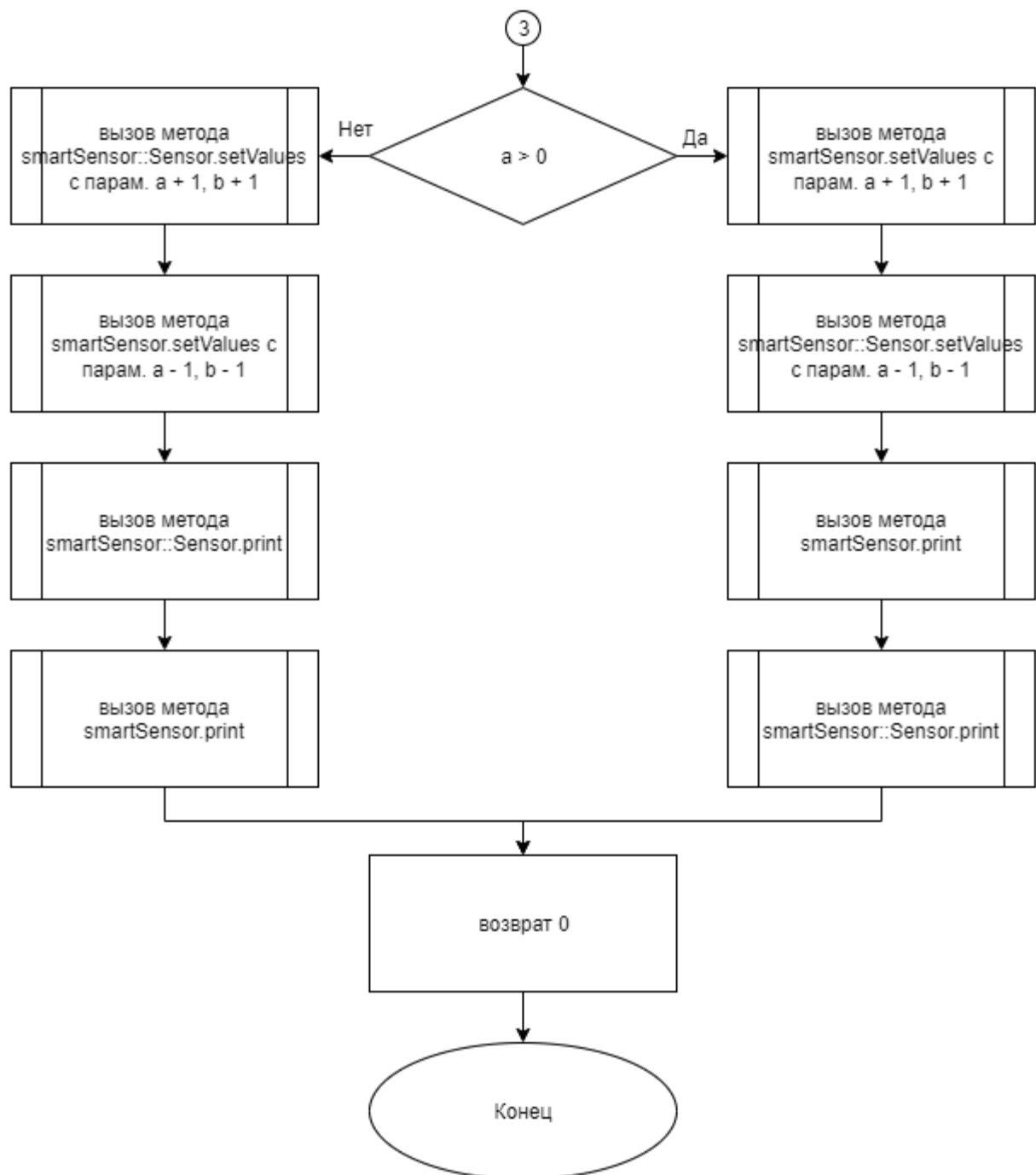


Рисунок 4 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include "SmartSensor.h"
#include <iostream>

int main()
{
    int var1, var2;
    std::cin >> var1 >> var2;
    SmartSensor smartSensor(var1, var2);

    smartSensor.Sensor::print();
    smartSensor.print();

    int number;
    std::cin >> number;
    smartSensor.publicValue = number;
    smartSensor.Sensor::publicValue = number;

    smartSensor.print();
    smartSensor.Sensor::print();

    int a, b;
    std::cin >> a >> b;

    if (a > 0) {
        smartSensor.setValues(a + 1, b + 1);
        smartSensor.Sensor::setValues(a - 1, b - 1);

        smartSensor.print();
        smartSensor.Sensor::print();
    } else {
        smartSensor.Sensor::setValues(a + 1, b + 1);
        smartSensor.setValues(a - 1, b - 1);

        smartSensor.Sensor::print();
        smartSensor.print();
    }
}
```

```
    return 0;
}
```

5.2 Файл Sensor.cpp

Листинг 2 – Sensor.cpp

```
#include "Sensor.h"
#include <iostream>

void Sensor::setPrivateValue(int value) {
    privateValue = value * 3;
}

Sensor::Sensor(int param1, int param2) {
    setValues(param1, param2);
}

void Sensor::setValues(int param1, int param2) {
    setPrivateValue(param1);
    publicValue = param2;
}

void Sensor::print() const {
    std::cout << "Sensor: " << privateValue << " " << publicValue <<
    std::endl;
}
```

5.3 Файл Sensor.h

Листинг 3 – Sensor.h

```
#ifndef __SENSOR__H
#define __SENSOR__H

class Sensor {
private:
    int privateValue;
    void setPrivateValue(int value);

public:
    int publicValue;
    Sensor(int param1, int param2);
    void setValues(int param1, int param2);
    void print() const;
};
```

```
};  
  
#endif
```

5.4 Файл SmartSensor.cpp

Листинг 4 – SmartSensor.cpp

```
#include "SmartSensor.h"  
#include <iostream>  
  
SmartSensor::SmartSensor(int param1, int param2)  
    : Sensor(param1, param2) {  
    setValues(param1, param2);  
}  
  
void SmartSensor::setValues(int param1, int param2) {  
    privateValue = param1;  
    publicValue = param2;  
}  
  
void SmartSensor::print() const {  
    std::cout << "SmartSensor: " << privateValue << " " << publicValue <<  
    std::endl;  
}
```

5.5 Файл SmartSensor.h

Листинг 5 – SmartSensor.h

```
#ifndef __SMARTSENSOR__H  
#define __SMARTSENSOR__H  
#include "Sensor.h"  
  
class SmartSensor : public Sensor {  
private:  
    int privateValue;  
  
public:  
    int publicValue;  
    SmartSensor(int param1, int param2);  
    void setValues(int param1, int param2);  
    void print() const;  
};
```

```
#endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 10.

Таблица 10 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 4 5 2 3	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 SmartSensor: 3 4 Sensor: 3 2	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 SmartSensor: 3 4 Sensor: 3 2
1 2 3 3 4	Sensor: 3 2 SmartSensor: 1 2 SmartSensor: 1 3 Sensor: 3 3 SmartSensor: 4 5 Sensor: 6 3	Sensor: 3 2 SmartSensor: 1 2 SmartSensor: 1 3 Sensor: 3 3 SmartSensor: 4 5 Sensor: 6 3
4 5 9 -2 3	Sensor: 12 5 SmartSensor: 4 5 SmartSensor: 4 9 Sensor: 12 9 Sensor: -3 4 SmartSensor: -3 2	Sensor: 12 5 SmartSensor: 4 5 SmartSensor: 4 9 Sensor: 12 9 Sensor: -3 4 SmartSensor: -3 2
0 0 0 0 0	Sensor: 0 0 SmartSensor: 0 0 SmartSensor: 0 0 Sensor: 0 0 Sensor: 3 1 SmartSensor: -1 -1	Sensor: 0 0 SmartSensor: 0 0 SmartSensor: 0 0 Sensor: 0 0 Sensor: 3 1 SmartSensor: -1 -1

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).